PostgreSQL Date & Time Cheat Sheet (Java 8+ java.time)

This guide focuses on mapping modern Java 8+ Date/Time API types to PostgreSQL column types using JDBC.

Quick Type Mapping Table

Java Type (java.time)	PostgreSQL Type	java.sql.Types Constant	Description
LocalDate	DATE	Types.DATE	Year, Month, Day (No time)
LocalTime	TIME	Types.TIME	Hour, Minute, Second, Nano (No date)
LocalDateTime	TIMESTAMP	Types.TIMESTAMP	Date + Time (No Timezone)
OffsetDateTime	TIMESTAMPTZ	Types.TIMESTAMP_WITH_TIMEZONE	Date + Time + Offset (UTC normalization)

1. LocalDate (DATE)

Use for birthdays, invoice dates, etc.

Setting Value (PreparedStatement)

Recommended (JDBC 4.2+): Use setObject. It avoids manual conversion.

```
LocalDate birthDate = entity.getBirthDate();

if (birthDate == null) {
    stmt.setNull(1, java.sql.Types.DATE);
} else {
    // Modern drivers handle LocalDate directly
    stmt.setObject(1, birthDate);
}
```

Legacy / Manual Conversion Way:

```
if (birthDate == null) {
   stmt.setNull(1, java.sql.Types.DATE);
} else {
   stmt.setDate(1, java.sql.Date.valueOf(birthDate));
}
```

Getting Value (ResultSet)

Recommended:

```
// getObject(index, Class) avoids null checks for the object itself
LocalDate date = rs.getObject("column_name", LocalDate.class);
entity.setBirthDate(date); // safe even if null
```

Legacy / Manual Conversion Way:

```
java.sql.Date sqlDate = rs.getDate("column_name");
if (sqlDate != null) {
   entity.setBirthDate(sqlDate.toLocalDate());
} else {
   entity.setBirthDate(null);
}
```

2. LocalDateTime (TIMESTAMP)

Use for created_at, updated_at, event start times (where timezone is implied or irrelevant).

Setting Value (PreparedStatement)

Recommended:

```
LocalDateTime createdAt = entity.getCreatedAt();

if (createdAt == null) {
   stmt.setNull(2, java.sql.Types.TIMESTAMP);
} else {
   stmt.setObject(2, createdAt);
```

```
}
```

Legacy / Manual Conversion Way:

```
if (createdAt == null) {
   stmt.setNull(2, java.sql.Types.TIMESTAMP);
} else {
   stmt.setTimestamp(2, java.sql.Timestamp.valueOf(createdAt));
}
```

Getting Value (ResultSet)

Recommended:

```
LocalDateTime ts = rs.getObject("column_name", LocalDateTime.class);
entity.setCreatedAt(ts);
```

Legacy / Manual Conversion Way:

```
java.sql.Timestamp timestamp = rs.getTimestamp("column_name");
if (timestamp != null) {
   entity.setCreatedAt(timestamp.toLocalDateTime());
} else {
   entity.setCreatedAt(null);
}
```

3. LocalTime (TIME)

Use for opening hours, daily schedules (without date).

Setting Value (PreparedStatement)

Recommended:

```
LocalTime openTime = entity.getOpenTime();

if (openTime == null) {
   stmt.setNull(3, java.sql.Types.TIME);
} else {
   stmt.setObject(3, openTime);
```

```
}
```

Legacy / Manual Conversion Way:

```
if (openTime == null) {
    stmt.setNull(3, java.sql.Types.TIME);
} else {
    stmt.setTime(3, java.sql.Time.valueOf(openTime));
}
```

Getting Value (ResultSet)

Recommended:

```
LocalTime time = rs.getObject("column_name", LocalTime.class);
entity.setOpenTime(time);
```

Legacy / Manual Conversion Way:

```
java.sql.Time sqlTime = rs.getTime("column_name");
if (sqlTime != null) {
    entity.setOpenTime(sqlTime.toLocalTime());
} else {
    entity.setOpenTime(null);
}
```

4. OffsetDateTime (TIMESTAMPTZ)

Use for precise instants in time (logs, transactions) where you need to preserve the exact point in UTC.

Note: PostgreSQL stores TIMESTAMPTZ normalized to UTC. When reading back, the JDBC driver typically returns it in the JVM's local time or UTC depending on configuration.

Setting Value

```
OffsetDateTime transactionTime = entity.getTransactionTime();

if (transactionTime == null) {
    stmt.setNull(4, java.sql.Types.TIMESTAMP_WITH_TIMEZONE);
```

```
} else {
    stmt.setObject(4, transactionTime);
}
```

Getting Value

```
OffsetDateTime odt = rs.getObject("column_name", OffsetDateTime.class);
entity.setTransactionTime(odt);
```

5. Summary of NULL Handling

When setting a value to NULL in PreparedStatement, you **must** specify the correct SQL type constant.

```
stmt.setNull(index, java.sql.Types.DATE);      // For LocalDate
stmt.setNull(index, java.sql.Types.TIMESTAMP); // For LocalDateTime
stmt.setNull(index, java.sql.Types.TIME);      // For LocalTime
```